

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CASE STUDY BASED ASSESSMENT

Course Code:	CSA0604	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Case Study Based Assessment
Weightage:	20% of CIA	Total Marks:	50
CO Statement:	<i>Apply Divide and Conquer technique to solve sorting problems and analyze algorithm efficiency (BL1, BL2, BL3)</i>		
Student Name:	G. Hari Prasad	Reg. No.:	192525273
Section / Batch:	B	Date:	05/08/2026

Question Type	Focus Area	Marks
Case Study Based Assessment	Analyze a real-world medical imaging scenario and apply Merge Sort — stability, divide and conquer, and time complexity	50
GRAND TOTAL:		50 Marks

SECTION A – CASE STUDY BASED ASSESSMENT

Instructions to Students

- Carefully analyze the given case study and identify the computational problem involved.
- Apply Merge Sort's divide-and-conquer approach and explain the process step by step.
- Implement and execute the algorithm to demonstrate stability, large-data handling, and accuracy.
- Analyze the time complexity and justify why Merge Sort is suitable for this scenario.
- Present code, actual program output, and a clear written analysis with proper technical reasoning.

Code of Conduct

I **G. Hari Prasad (192525273)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *G. Hari Prasad*

Q36. Case Study: Medical Imaging Data Sorting (Merge Sort)

A medical imaging system sorts patient scan data based on time and priority. Analyze how Merge Sort ensures stable sorting. Identify issues such as large data size and accuracy. Apply divide and conquer concepts to explain the process. Analyze time complexity and justify its use.

Program Python Code:

```
import time
import random
from collections import defaultdict

def merge(left, right, key):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(records, key=lambda r: r['priority']):
    if len(records) <= 1:
        return records
    mid = len(records) // 2
    left = merge_sort(records[:mid], key)
    right = merge_sort(records[mid:], key)
    return merge(left, right, key)

# ---- Stability demonstration ----
scans = [
    {'scan_id': 'S101', 'patient': 'Ravi', 'time': '09:00', 'priority': 2},
    {'scan_id': 'S102', 'patient': 'Meena', 'time': '09:05', 'priority': 1},
    {'scan_id': 'S103', 'patient': 'Arjun', 'time': '09:10', 'priority': 2},
    {'scan_id': 'S104', 'patient': 'Divya', 'time': '09:15', 'priority': 3},
    {'scan_id': 'S105', 'patient': 'Kiran', 'time': '09:20', 'priority': 1},
    {'scan_id': 'S106', 'patient': 'Suresh', 'time': '09:25', 'priority': 2},
```

```
]

print("\nOriginal order (by scan admission time):")
for r in scans:
    print(f" {r['scan_id']} {r['patient']:<8} time={r['time']} priority={r['priority']}")

sorted_scans = merge_sort(scans)

print("\nSorted by priority using Merge Sort:")
for r in sorted_scans:
    print(f" {r['scan_id']} {r['patient']:<8} time={r['time']} priority={r['priority']}")

orig_groups = defaultdict(list)
for r in scans:
    orig_groups[r['priority']].append(r['scan_id'])
sorted_groups = defaultdict(list)
for r in sorted_scans:
    sorted_groups[r['priority']].append(r['scan_id'])

stable = all(orig_groups[p] == sorted_groups[p] for p in orig_groups)
print(f"\nStability check -> relative time-order preserved within each priority group: {stable}")
for p in sorted(orig_groups):
    print(f" Priority {p}: original order {orig_groups[p]} -> sorted-group order {sorted_groups[p]}")

# ---- Time complexity measurement on larger datasets ----
def generate_scans(n):
    return [{'scan_id': f'S{i}', 'priority': random.randint(1, 5)} for i in range(n)]

print("\nTime Complexity Measurement (random priority data):")
random.seed(7)
for n in [2000, 20000, 200000]:
    data = generate_scans(n)
    t0 = time.perf_counter()
    merge_sort(data)
    t1 = time.perf_counter()
    print(f" n = {n:>7} -> Execution Time: {(t1 - t0) * 1000:.3f} ms")
```

Output (Actual Execution):

```

PS C:\Users\hariprasad\Desktop\DAA> python mergesort_medical.py
Original order (by scan admission time):
S101 Ravi      time=09:00  priority=2
S102 Meena    time=09:05  priority=1
S103 Arjun    time=09:10  priority=2
S104 Divya    time=09:15  priority=3
S105 Kiran    time=09:20  priority=1
S106 Suresh   time=09:25  priority=2

Sorted by priority using Merge Sort:
S102 Meena    time=09:05  priority=1
S105 Kiran    time=09:20  priority=1
S101 Ravi     time=09:00  priority=2
S103 Arjun    time=09:10  priority=2
S106 Suresh   time=09:25  priority=2
S104 Divya    time=09:15  priority=3

Stability check -> relative time-order preserved within each priority group: True
Priority 1: original order ['S102', 'S105'] -> sorted-group order ['S102', 'S105']
Priority 2: original order ['S101', 'S103', 'S106'] -> sorted-group order ['S101', 'S103', 'S106']
Priority 3: original order ['S104'] -> sorted-group order ['S104']

Time Complexity Measurement (random priority data):
n = 2000 -> Execution Time: 5.187 ms
n = 20000 -> Execution Time: 68.416 ms
n = 200000 -> Execution Time: 927.106 ms
PS C:\Users\hariprasad\Desktop\DAA>

```

a) How Merge Sort Ensures Stable Sorting:

A sort is called stable if two records with equal keys retain their original relative order after sorting. In the medical imaging system, several scans can share the same priority level, so the tie-breaker (their original chronological order) must not be disturbed — a high-priority scan queued earlier must still come before one queued later at the same priority.

Merge Sort achieves this in the `merge()` step, where the comparison is written as `"if key(left[i]) <= key(right[j])"` rather than a strict `"<"`. Because the left half always contains elements that appeared earlier in the original array than the right half, using `<=` means that whenever priorities are equal, the element from the left half (the earlier one) is always copied into the result first. The executed output above confirms this directly: Priority 1 scans kept the order [S102, S105], Priority 2 scans kept [S101, S103, S106], and the stability check printed `True` — proving no reordering occurred among equal-priority scans.

b) Issues Identified: Large Data Size and Accuracy:

Large data size: A real hospital network can generate very large volumes of scan metadata (CT, MRI, X-ray sessions) continuously. Standard Merge Sort requires $O(n)$ auxiliary memory for the temporary arrays created during merging, which the results table below shows growing with n ; for extremely large datasets that exceed available RAM, an external merge sort (sorting in chunks on disk and merging the sorted chunks) becomes necessary, since in-memory recursive Merge Sort alone would exhaust memory or degrade performance.

Accuracy: Correct sorting depends entirely on the priority and timestamp fields being clean and consistent. Missing priority values, inconsistent timestamp formats, or duplicate scan IDs can silently produce a technically "sorted" but clinically wrong ordering. Because Merge Sort is comparison-based, its correctness is only as good as the comparator (key function); the system must validate/normalize priority and time fields before sorting so that equal-priority ties are broken purely by genuine arrival time, not by corrupted data.

c) Divide and Conquer Explanation:

Divide: The list of scan records is repeatedly split at the midpoint into two halves until each sublist contains a single record (a single record is trivially sorted). **Conquer:** Each half is recursively sorted using the same Merge Sort procedure, breaking the large sorting problem into smaller identical subproblems. **Combine:** The two sorted halves are merged back together in the `merge()` function, comparing the front elements of each half and appending the smaller (or equal, from the left half) one at a time — reconstructing a single sorted sequence from the two sorted halves in linear time relative to their combined size.

In this case study, the recursive calls in `merge_sort()` perform the divide step, the base case `"if len(records) <= 1: return records"` is the trivial conquer step, and `merge()` performs the combine step that also guarantees stability.

d) Time Complexity Analysis and Justification:

At every level of recursion, the divide step splits the data in half, producing $\log_2 n$ levels; at each level, the merge step processes all n elements once, giving a total time complexity of $O(n \log n)$ in the best, average, and worst cases alike — unlike Quick Sort, which can degrade to $O(n^2)$ on already-sorted or adversarial input. The space complexity is $O(n)$ due to the temporary arrays created during merging.

Input Size (n)	Execution Time (ms)	Approx. $n \log_2 n$ growth
2,000	5.187	$n \log n$ baseline
20,000	68.416	$\approx 13.2\times$ (theory $\approx 13.0\times$)
200,000	927.106	$\approx 13.6\times$ (theory $\approx 12.3\times$)

The measured execution times closely track the theoretical $n \log n$ growth rate: increasing n by $10\times$ (from 2,000 to 20,000) increased runtime by about $13.2\times$, and scaling to 200,000 kept the observed growth close to the $\approx 12\text{--}13\times$ predicted by $n \log n$ scaling — confirming Merge Sort's predictable $O(n \log n)$ behaviour on real execution.

Justification for use: Merge Sort is well suited to this medical imaging scenario because (1) it is stable, so scans with equal priority remain ordered by genuine arrival time without any extra bookkeeping; (2) its $O(n \log n)$ time complexity is guaranteed regardless of how the incoming data is ordered, giving predictable, consistent performance that is important in a clinical setting where turnaround time matters; and (3) its divide-and-conquer structure parallelizes and scales cleanly to large datasets (and can be extended to external/merge-on-disk sorting when data volume exceeds memory), making it a dependable choice over unstable or worst-case-fragile alternatives such as Quick Sort or Selection Sort for this use case.